

Lecture 07

# **RISC-V: Single-Cycle and Multi-Cycle Implementations**

Instructor: Dr. Muhammad Imran



Computer Architecture | RISC-V Implementation

# Acknowledgement

---

- Content from following has been used in these lectures
  - Computer Organization and Design, RISC-V 2<sup>nd</sup> Edition, Patterson and Hennessy
  - CS 61C: Great Ideas in Computer Architecture (Machine Structures), UC Berkeley

# Contents

---

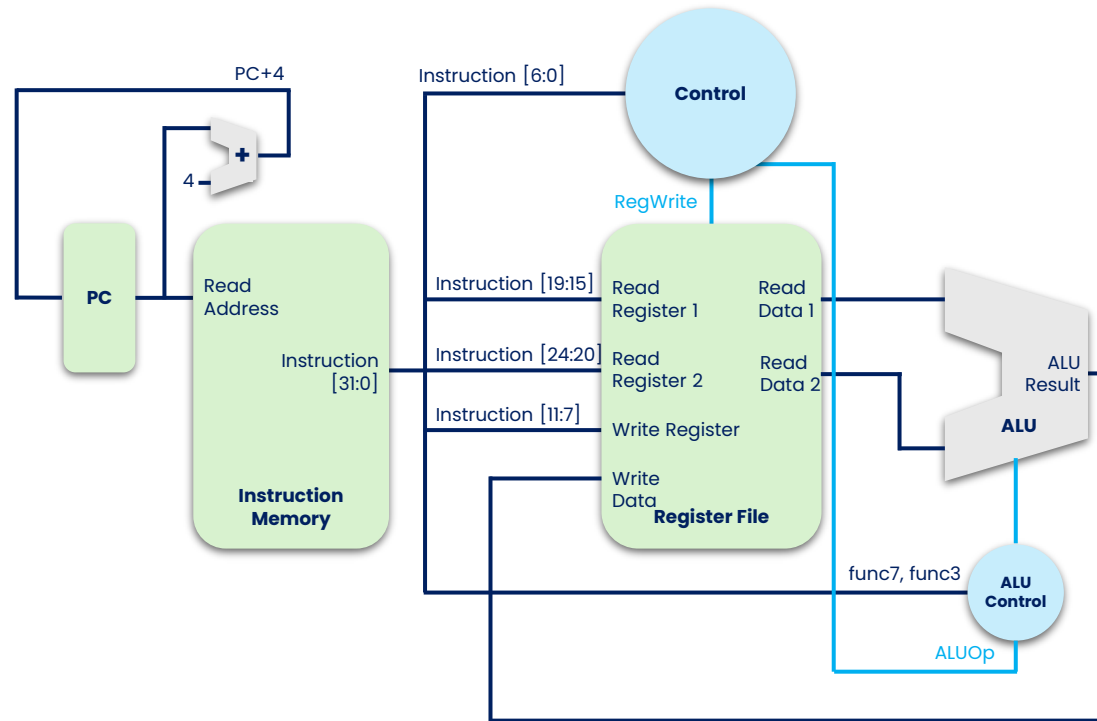
- Building Single-Cycle Microarchitecture
  - Arithmetic and Logical Instructions
  - ALU Implementation
  - Data Transfer Instructions
  - Branches
  - Implementing Control Logic
- RISC-V Multicycle Implementation
  - Datapath
  - Control

# Implementing Arithmetic and Logical instructions ...



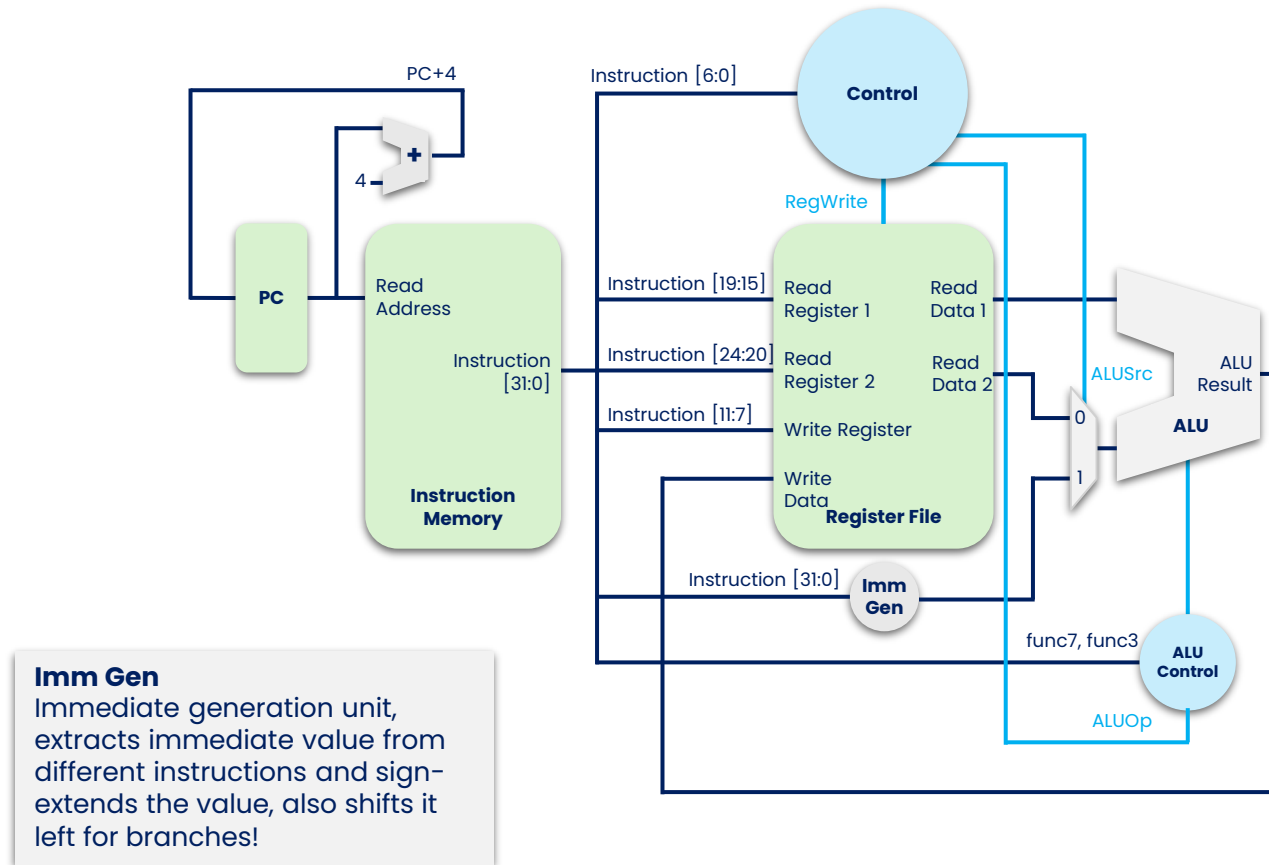
# Arithmetic and Logical Instructions – R-Type

- add, sub, and, sll etc.



# Arithmetic and Logical Instructions – Adding I-Type

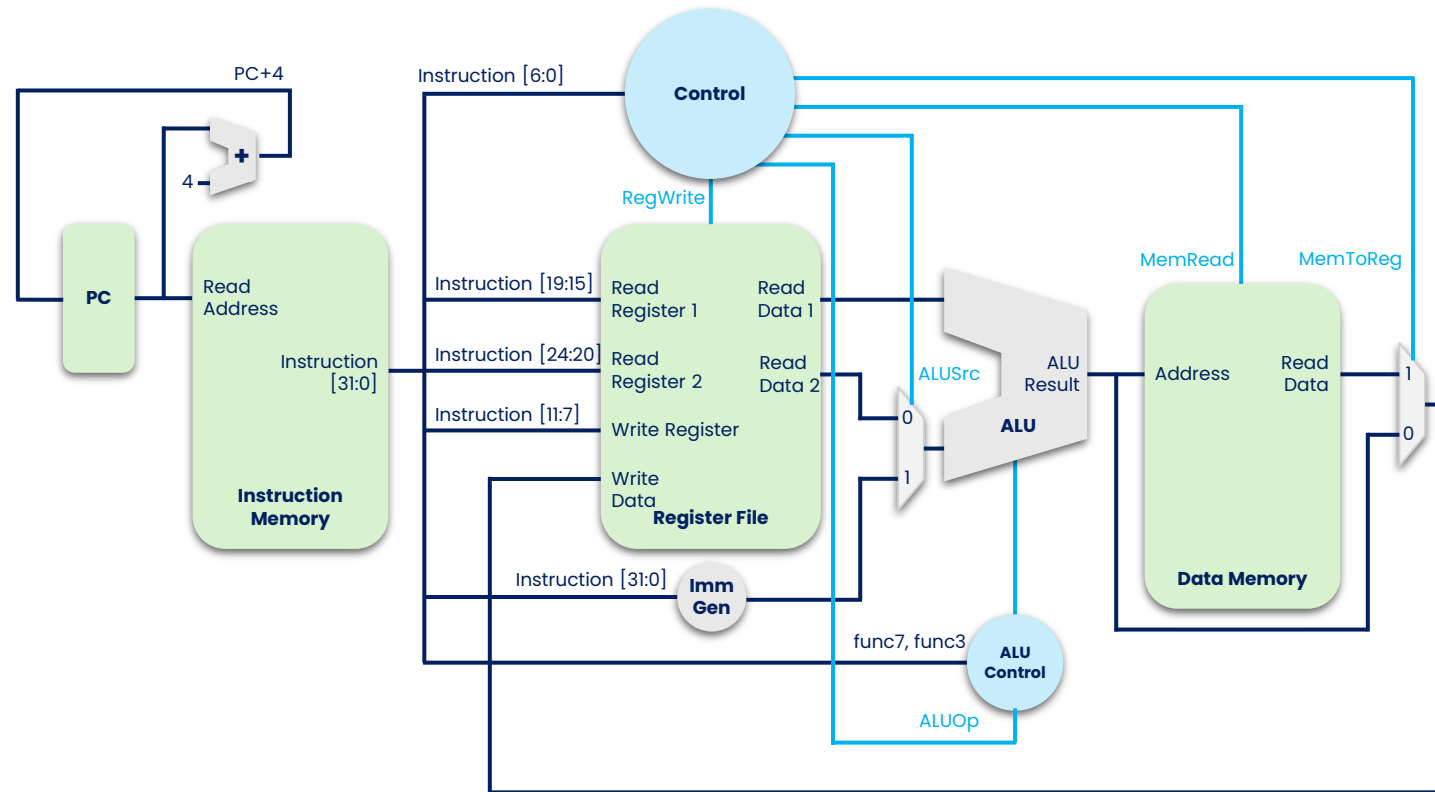
- add, addi, and, andi, sll, slli etc.



# Extending the microarchitecture to handle Data Transfer instructions



# Load Instruction





# Data Memory Limitations

- To simplify its circuitry, DMEM can ONLY access 32 bits at a time
  - Regardless of what load/store instruction you have, dataW and dataR will always be 32-bits long.
- Even further, many RISC-V DMEMs have a further restriction that the DMEM can only access addresses that are a multiple of 4.
  - Ex. You can access address **0xFFFF FFC** to get the data from **0xFFFF FFC** to **0xFFFF FFF**, but not **0xFFFF FFD**

| Instruction                   | Name                         | Description                                                         | Type | Opcode   | Funct3 | Funct7 |
|-------------------------------|------------------------------|---------------------------------------------------------------------|------|----------|--------|--------|
| <b>lb</b> <b>rd imm(rs1)</b>  | Load Byte                    | rd = 1 byte of memory at address rs1 + imm, sign-extended           | I    | 000 0011 | 000    |        |
| <b>lbu</b> <b>rd imm(rs1)</b> | Load Byte<br>(Unsigned)      | rd = 1 byte of memory at address rs1 + imm, zero-extended           | I    | 000 0011 | 100    |        |
| <b>lh</b> <b>rd imm(rs1)</b>  | Load Half-word               | rd = 2 bytes of memory starting at address rs1 + imm, sign-extended | I    | 000 0011 | 001    |        |
| <b>lhu</b> <b>rd imm(rs1)</b> | Load Half-word<br>(Unsigned) | rd = 2 bytes of memory starting at address rs1 + imm, zero-extended | I    | 000 0011 | 101    |        |
| <b>lw</b> <b>rd imm(rs1)</b>  | Load Word                    | rd = 4 bytes of memory starting at address rs1 + imm                | I    | 000 0011 | 010    |        |

# Data Memory Limitations affect Programming

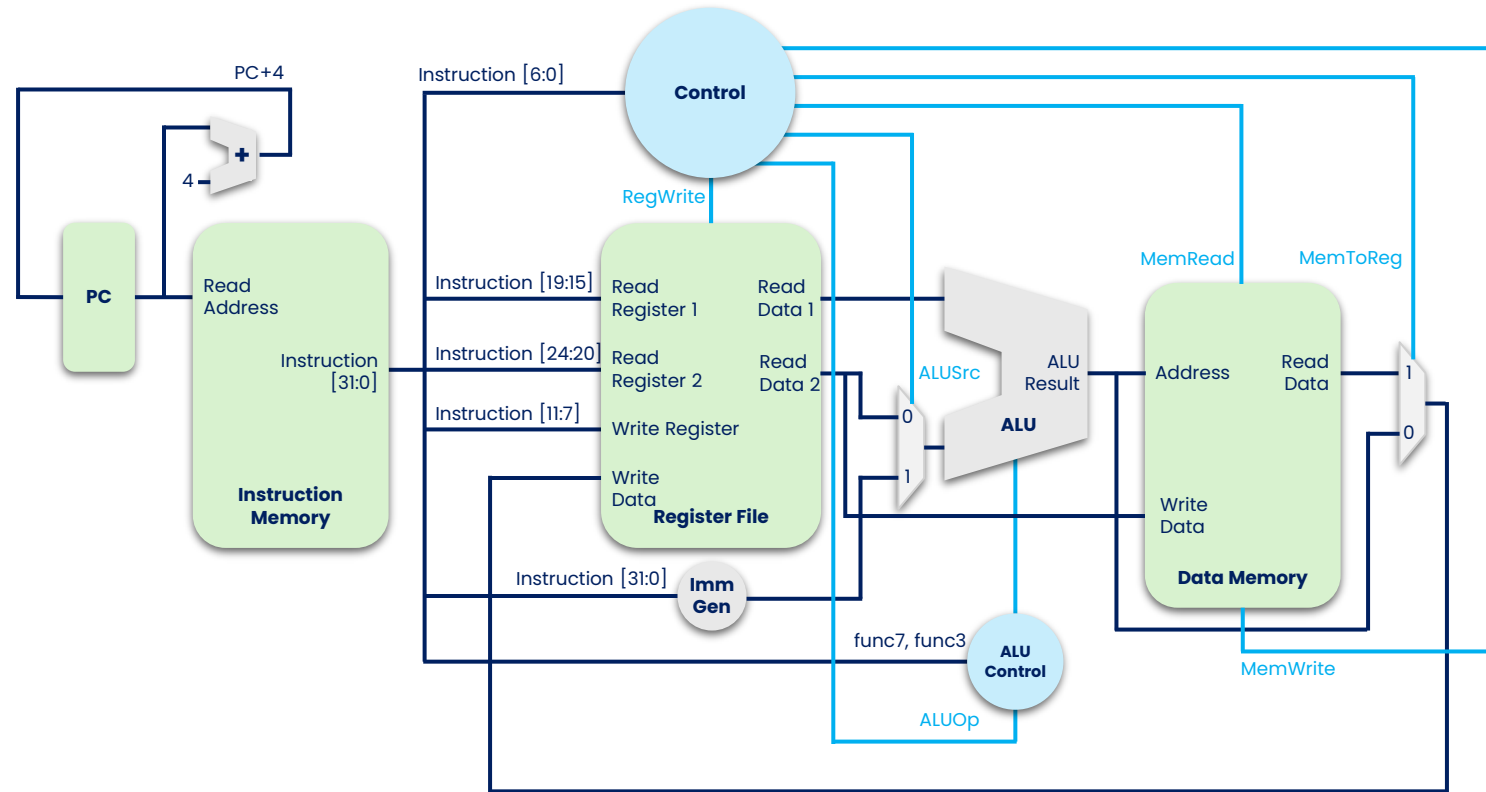
- Because of this inherent limitation, RISC-V often mandates that loads/stores happen on aligned addresses:
  - lw/sw must be run on an address that's a multiple of 4
    - ex. lw s0 3(sp) is illegal unless sp is 1 mod 4
  - lh/sh/lhu must be run on an address that's a multiple of 2
  - lb/sb/lbu can be run on any address
- Other accesses yield undefined behavior: the CPU can do anything if someone tries an unaligned access
- It's possible to create an unaligned access (pseudoinstruction-style) using only aligned accesses, but it's much slower.
  - Venus lets you use unaligned accesses despite this
  - Even in x86, which allows unaligned accesses, they take much longer to run
- This is why structs tend to place their components in a way that they're word-aligned, even if that wastes some space.

# Data Memory Limitations affect our Circuit – Loads

---

- General procedure to handle loads:
  1. Send into the DMEM the address of the word containing your desired data
    - Since only aligned accesses are allowed, no load will ever require data from two different 4-byte blocks
    - Ex. The byte starting at **0xDEADBEEF** is contained with the 4-byte word starting at **0xDEADBEEC**
  2. Split the retrieved word into bytes/halfwords
  3. Select the appropriate byte/halfword using **address%4** and mux
  4. Sign- or Zero-extend the resulting data to a full 32 bits

# Store Instruction



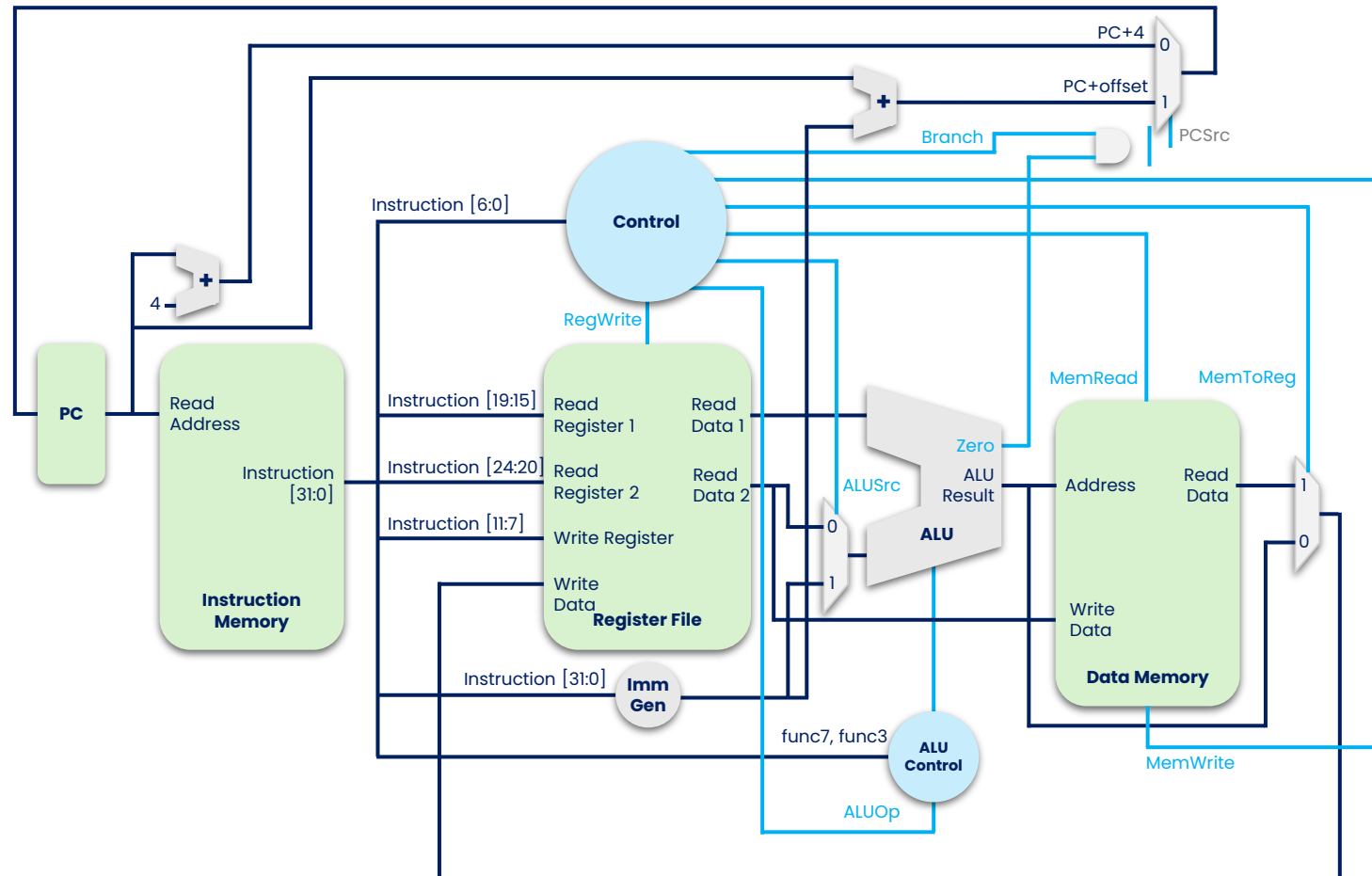
# Data Memory Limitations affect our Circuit – Stores

---

- As with loads, DMEM's limitations affect our circuit design.
- General approach:
  1. Determine which word contains the data we will store
    - As with loads, no store to two separate words
  2. Manipulate the data to align with the right bytes
    - Left-shifts according to the endianness
  3. Use separate read/write bits for each byte, so you don't overwrite unnecessary data

# Extending the microarchitecture to handle branch instructions

# Branch (beq) Instruction



# Implementing ImmGen





# How to make the ImmGen ?

---

- **Option 1:**

- Make a circuit for each instruction format, then mux the results with ImmSel
- Ugly, but it works

- **Option 2:**

- Look for patterns in the instruction format
- Takes time, but gives much greater insight into why instruction formats were designed this way

# Instruction Formats

|           | 31                   | 25 24    | 20 19 | 15 14  | 12 11       | 7 6    | 0      |
|-----------|----------------------|----------|-------|--------|-------------|--------|--------|
| <b>R</b>  | funct7               | rs2      | rs1   | funct3 | rd          | opcode |        |
| <b>I</b>  | imm[11:0]            |          |       | rs1    | funct3      | rd     | opcode |
| <b>I*</b> | funct7               | imm[4:0] | rs1   | funct3 | rd          | opcode |        |
| <b>S</b>  | imm[11:5]            | rs2      | rs1   | funct3 | imm[4:0]    | opcode |        |
| <b>B</b>  | imm[12 10:5]         | rs2      | rs1   | funct3 | imm[4:1 11] | opcode |        |
| <b>U</b>  | imm[31:12]           |          |       |        |             | rd     | opcode |
| <b>J</b>  | imm[20 10:1 11 9:12] |          |       |        |             |        |        |

# Tracking all Immediate Bits in Each Format

- Immediate has been placed
  - To match maximum bit positions in different formats
  - Exploiting this property can simplify **ImmGen** implementation logic
  - Just requires a bunch of muxes controlled by bits from opcode

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| <b>Imm</b> | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |    |  |  |  |  |  |  |  |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|----|--|--|--|--|--|--|--|
| <b>I</b> | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |   |   |   |   |    |  |  |  |  |  |  |  |
| <b>S</b> | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4 | 3 | 2 | 1 | 0  |  |  |  |  |  |  |  |
| <b>B</b> | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4 | 3 | 2 | 1 | 11 |  |  |  |  |  |  |  |
| <b>U</b> | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |   |   |   |   |    |  |  |  |  |  |  |  |
| <b>J</b> | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |   |   |   |   |    |  |  |  |  |  |  |  |

# Tracking all Immediate Bits in Each Format

- **Bit 0 of imm:**

- Bit 20 of inst if I type
- Bit 7 of inst if S type
- Always 0 if B/U/J type

|     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |    |  |  |  |  |  |  |  |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|----|--|--|--|--|--|--|--|
| <b>I</b> | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |   |   |   |   |    |  |  |  |  |  |  |  |
| <b>S</b> | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4 | 3 | 2 | 1 | 0  |  |  |  |  |  |  |  |
| <b>B</b> | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4 | 3 | 2 | 1 | 11 |  |  |  |  |  |  |  |
| <b>U</b> | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |   |   |   |   |    |  |  |  |  |  |  |  |
| <b>J</b> | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |   |   |   |   |    |  |  |  |  |  |  |  |

# Tracking all Immediate Bits in Each Format

- **Bits 4–1 of imm:**

- Bits 24–21 of inst if I/J type
- Bits 11–8 of inst if S/B type
- Always 0 if U

- **Only found in two parts of the instruction, and these 4 bits are always together**

|     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|

|   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |    |  |  |  |  |  |  |  |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|----|--|--|--|--|--|--|--|
| I | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |   |   |   |   |    |  |  |  |  |  |  |  |
| S | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4 | 3 | 2 | 1 | 0  |  |  |  |  |  |  |  |
| B | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4 | 3 | 2 | 1 | 11 |  |  |  |  |  |  |  |
| U | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |   |   |   |   |    |  |  |  |  |  |  |  |
| J | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |   |   |   |   |    |  |  |  |  |  |  |  |

# Tracking all Immediate Bits in Each Format

- **Bits 10–5 of imm:**

- Bits 30–25 of inst if I/S/B/J type
- Always 0 if U type.

- **Only found in one part of the instruction, even though 4 different instruction formats use it!**

| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |  |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|----|---|---|---|---|---|---|---|--|
| I   | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |    |    |   |   |    |   |   |   |   |   |   |   |  |
| S   | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 0  |   |   |   |   |   |   |   |  |
| B   | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 11 |   |   |   |   |   |   |   |  |
| U   | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |  |
| J   | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |  |

# Tracking all Immediate Bits in Each Format

- **MSB of imm:**

- is ALWAYS the MSB of the instruction.
- Sign-extending is just copying the MSB, and almost all RISC-V immediates sign-extend.
- **To sign-extend, we just take the MSB of the instruction.**

| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|----|---|---|---|---|---|---|---|
| I   | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |    |    |   |   |    |   |   |   |   |   |   |   |
| S   | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 0  |   |   |   |   |   |   |   |
| B   | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 11 |   |   |   |   |   |   |   |
| U   | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |
| J   | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |

# Tracking all Immediate Bits in Each Format

- **Bits 30–21 and 19–12 of immediate:**
  - Always the corresponding bit in the instruction **if part of the format**
  - **and bit 31** otherwise.
- **Only two options for these bits as well**

| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|----|---|---|---|---|---|---|---|
| I   | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |    |    |   |   |    |   |   |   |   |   |   |   |
| S   | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 0  |   |   |   |   |   |   |   |
| B   | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 11 |   |   |   |   |   |   |   |
| U   | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |
| J   | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |



# Tracking all Immediate Bits in Each Format

- **Bits 20 and 11 of immediate:**
  - Kind of put in random spots where there's space
  - Bit 20 needs a 3-way mux
  - Bit 11 needs a 4-way mux

| Imm | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|----|---|---|---|---|---|---|---|
| I   | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |    |    |    |    |    |    |    |    |    |   |   |    |   |   |   |   |   |   |   |
| S   | 11 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 0  |   |   |   |   |   |   |   |
| B   | 12 | 10 | 9  | 8  | 7  | 6  | 5  |    |    |    |    |    |    |    |    |    |    |    |    |    | 4  | 3  | 2 | 1 | 11 |   |   |   |   |   |   |   |
| U   | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |
| J   | 20 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 11 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 |    |    |   |   |    |   |   |   |   |   |   |   |

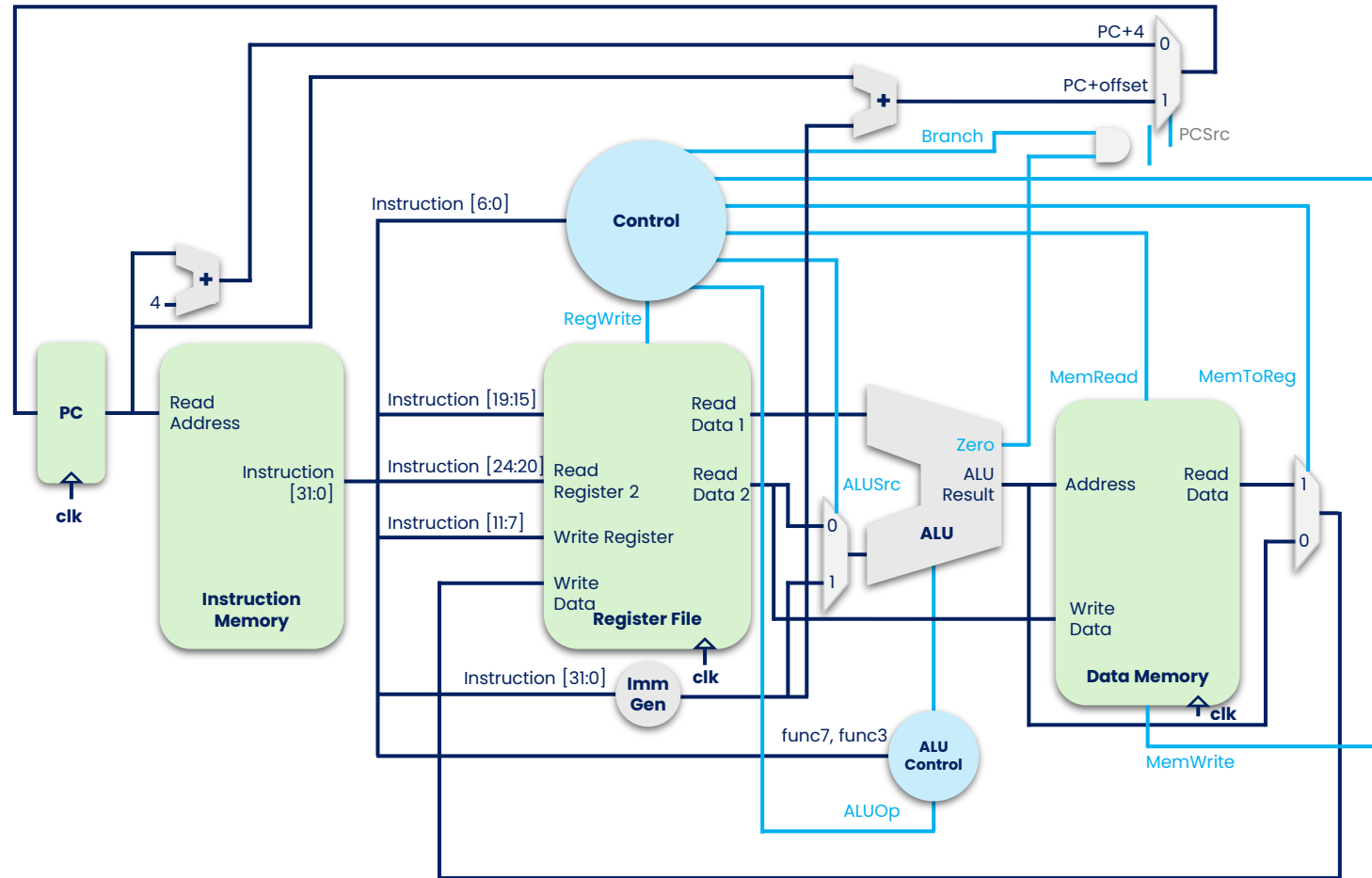
# How to make the ImmGen ?

---

- Because of how all instruction formats were chosen, there are a lot of underlying patterns in how immediates are stored.
  - By looking for these patterns, the ImmGen can be greatly simplified (in terms of total number of logic gates)
- 6-select 32-bit mux = 640 logic gates (counting 2-input OR and ANDs, and 1-input NOTs)
  - Using just muxes for each set of bits, you can get this down to 160 logic gates
  - **Con:** Makes it harder to extend your CPU. Often not worth it to optimize this too much

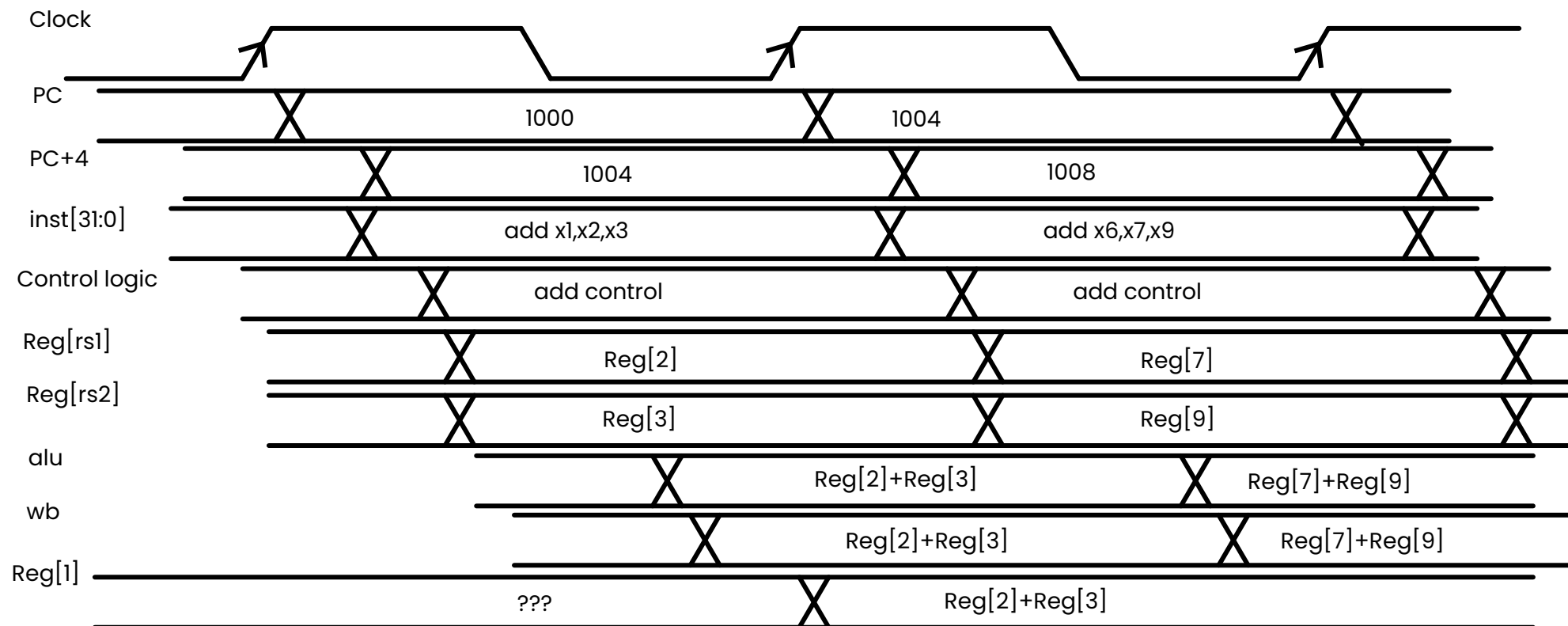
# Instruction Timing

# Example: **add** Timing

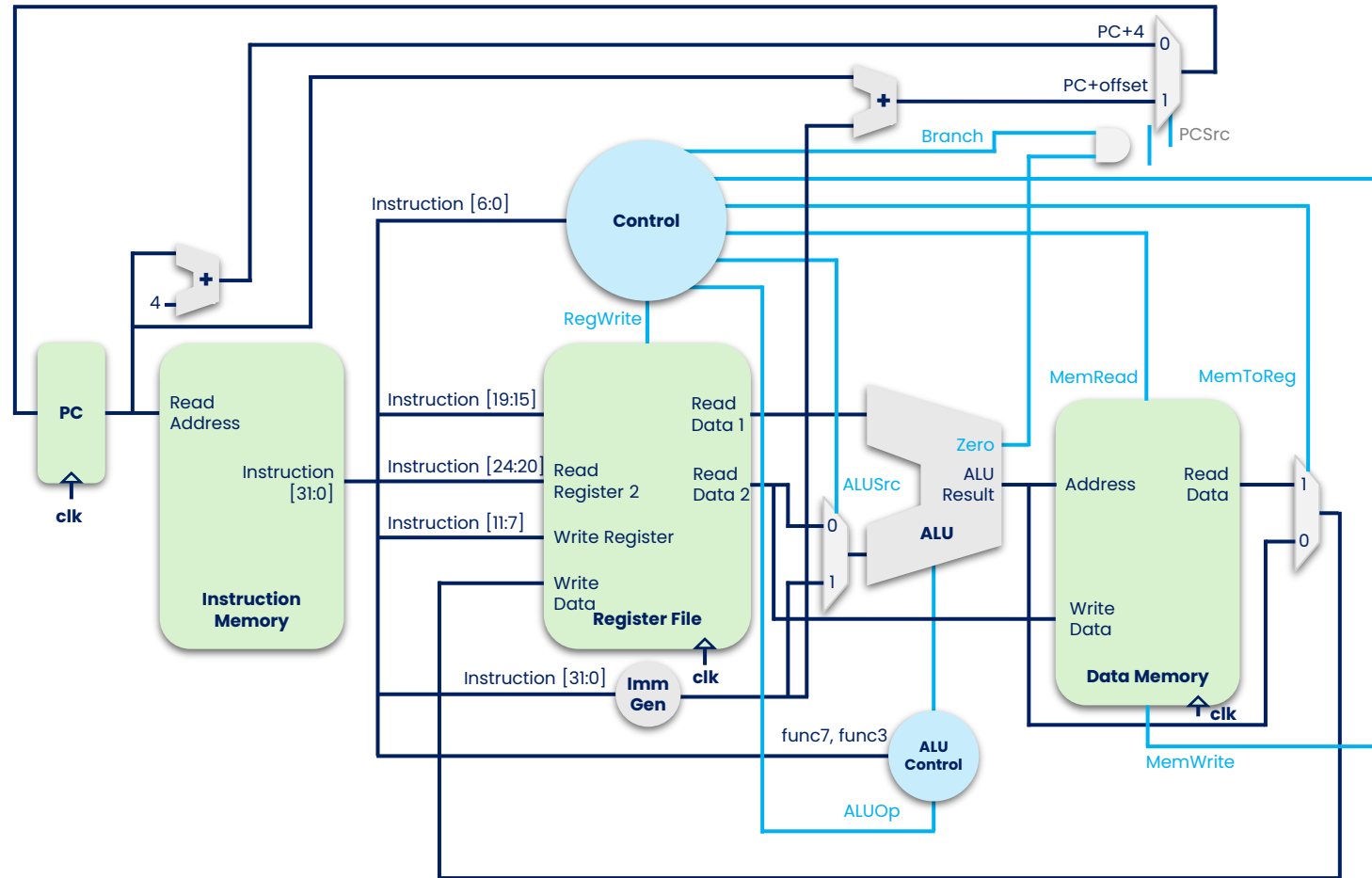


$$\begin{aligned}
 \text{Critical path delay} &= t_{clk-q} + \max\{t_{add} + t_{mux}, t_{imem} + t_{imm} + t_{add} + t_{mux}, t_{imem} + t_{reg} + t_{mux} + t_{alu} + t_{mux}\} + t_{setup} \\
 &= t_{clk-q} + t_{imem} + t_{reg} + t_{mux} + t_{alu} + t_{mux} + t_{setup}
 \end{aligned}$$

# Example: **add** Timing



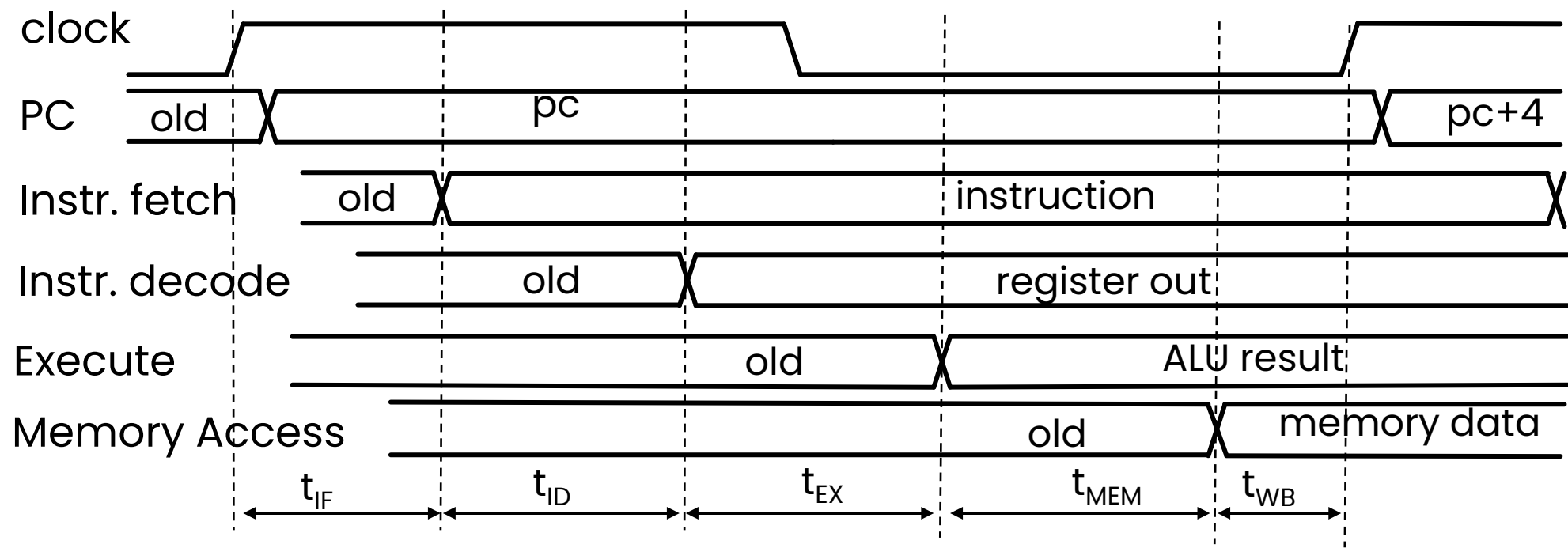
# Example: **lw** Timing



$$\begin{aligned}
 \text{Critical path delay} &= t_{clk-q} + \max\{t_{add} + t_{mux}, t_{imem} + t_{imm} + t_{add} + t_{mux}, \textcolor{red}{t_{imem} + t_{reg} + t_{mux} + t_{alu} + t_{dmem} + t_{mux}}\} + t_{setup} \\
 &= t_{clk-q} + \textcolor{red}{t_{imem} + t_{reg} + t_{mux} + t_{alu} + t_{dmem} + t_{mux}} + t_{setup}
 \end{aligned}$$

# Instruction Timing

| IF     | ID       | EX     | MEM    | WB     | Total  |
|--------|----------|--------|--------|--------|--------|
| I-MEM  | Reg Read | ALU    | D-MEM  | Reg W  |        |
| 200 ps | 100 ps   | 200 ps | 200 ps | 100 ps | 800 ps |



# Instruction Timing

- Maximum clock frequency
  - $f_{\max} = 1/800\text{ps} = 1.25 \text{ GHz}$
- Most blocks idle most of the time
  - E.g.  $f_{\max, \text{ALU}} = 1/200\text{ps} = 5 \text{ GHz!}$

| Instr      | IF = 200ps | ID = 100ps | ALU = 200ps | MEM=200ps | WB = 100ps | Total |
|------------|------------|------------|-------------|-----------|------------|-------|
| <b>add</b> | X          | X          | X           |           | X          | 600ps |
| <b>beq</b> | X          | X          | X           |           |            | 500ps |
| <b>jal</b> | X          | X          | X           |           |            | 500ps |
| <b>lw</b>  | X          | X          | X           | X         | X          | 800ps |
| <b>sw</b>  | X          | X          | X           | X         |            | 700ps |



# Implementing Control Logic



# The ALU Control

- ALU Control Lines for different operations

| ALU control lines | Function |
|-------------------|----------|
| 0000              | AND      |
| 0001              | OR       |
| 0010              | add      |
| 0110              | subtract |

- ALU is used for:
  - Arithmetic-logic operations in R-type instructions!
  - Address calculation (base+offset) in loads/stores
  - Comparing operands in branch instructions
    - Subtract and check zero result signal to test for equality!

# The ALU Control

---

- ALU Control Unit
  - Input
    - funt7 and funct3 bits
    - 2-bit control signal ALUOp
      - ALUOp is generated by main control unit using **OpCode Bits**
      - Such implementation simplifies main control unit!
      - ALUOp Value
        - 00: Load / Store
        - 01: beq
        - 10: R-type Instruction (add, sub, and, or)
      - ALUOp is generated based on type of instruction!
  - Output
    - ALU Control Lines

# The ALU Control

- ALU Control Signals

| Instruction opcode | ALUOp | Operation       | Funct7 field | Funct3 field | Desired ALU action | ALU control input |
|--------------------|-------|-----------------|--------------|--------------|--------------------|-------------------|
| lw                 | 00    | load word       | XXXXXXX      | XXX          | add                | 0010              |
| sw                 | 00    | store word      | XXXXXXX      | XXX          | add                | 0010              |
| beq                | 01    | branch if equal | XXXXXXX      | XXX          | subtract           | 0110              |
| R-type             | 10    | add             | 0000000      | 000          | add                | 0010              |
| R-type             | 10    | sub             | 0100000      | 000          | subtract           | 0110              |
| R-type             | 10    | and             | 0000000      | 111          | AND                | 0000              |
| R-type             | 10    | or              | 0000000      | 110          | OR                 | 0001              |

# The ALU Control

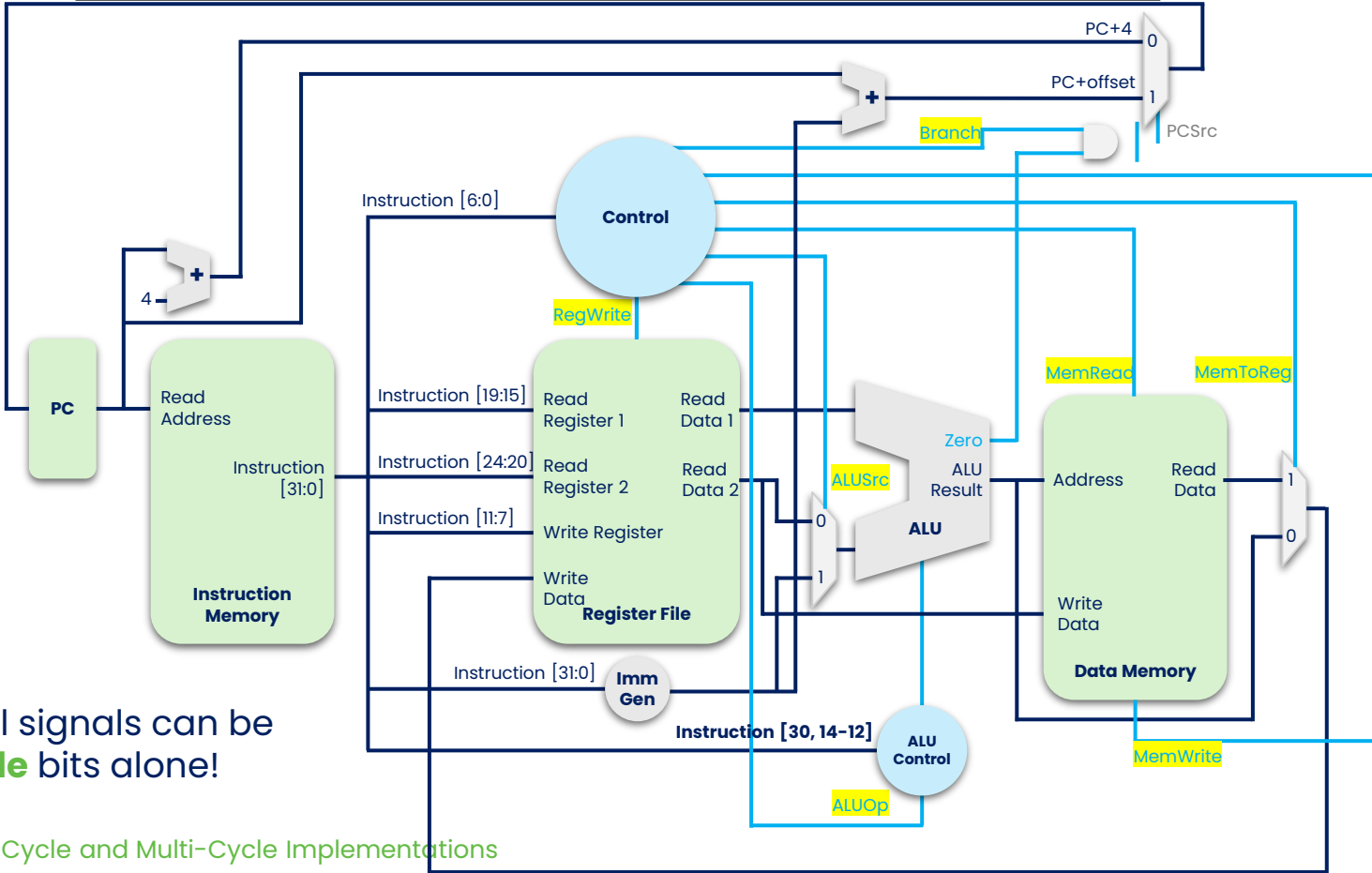
- Truth Table to implement logic for ALU Control Lines

| ALUOp  |        | Funct7 field |       |       |       |       |       |       | Funct3 field |       |       | Operation |
|--------|--------|--------------|-------|-------|-------|-------|-------|-------|--------------|-------|-------|-----------|
| ALUOp1 | ALUOp0 | I[31]        | I[30] | I[29] | I[28] | I[27] | I[26] | I[25] | I[14]        | I[13] | I[12] |           |
| 0      | 0      | X            | X     | X     | X     | X     | X     | X     | X            | X     | X     | 0010      |
| X      | 1      | X            | X     | X     | X     | X     | X     | X     | X            | X     | X     | 0110      |
| 1      | X      | 0            | 0     | 0     | 0     | 0     | 0     | 0     | 0            | 0     | 0     | 0010      |
| 1      | X      | 0            | 1     | 0     | 0     | 0     | 0     | 0     | 0            | 0     | 0     | 0110      |
| 1      | X      | 0            | 0     | 0     | 0     | 0     | 0     | 0     | 1            | 1     | 1     | 0000      |
| 1      | X      | 0            | 0     | 0     | 0     | 0     | 0     | 0     | 1            | 1     | 0     | 0001      |

- Only bits 30, 14, 13, and 12 are needed from function fields!
- Can you derive the logic circuitry for ALU control lines?

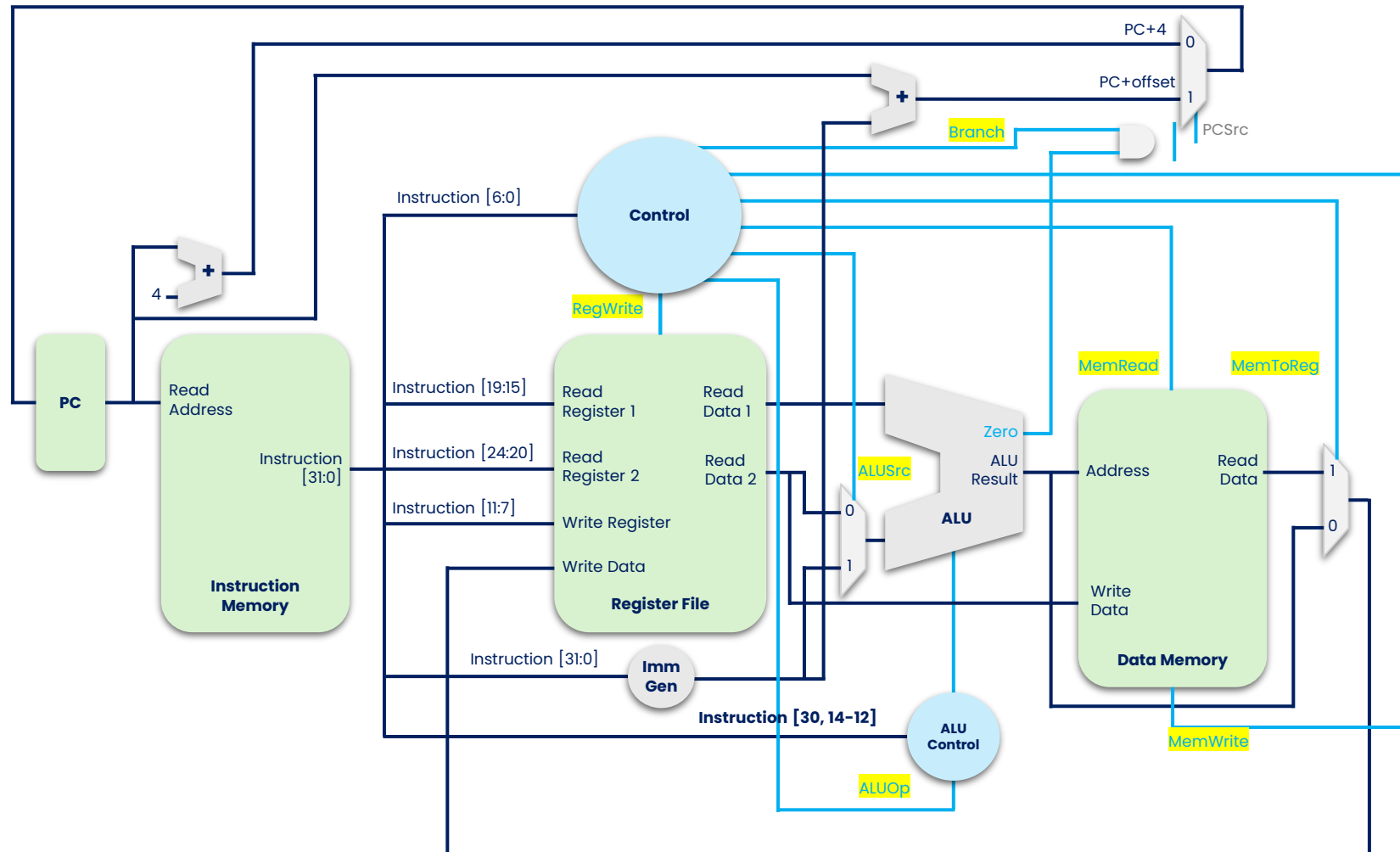
# Identifying Main Control Signals

| Instruction | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | ALUOp1 | ALUOp0 |
|-------------|--------|-----------|-----------|----------|-----------|--------|--------|--------|
| R-format    | 0      | 0         | 1         | 0        | 0         | 0      | 1      | 0      |
| lw          | 1      | 1         | 1         | 1        | 0         | 0      | 0      | 0      |
| sw          | 1      | X         | 0         | 0        | 1         | 0      | 0      | 0      |
| beq         | 0      | X         | 0         | 0        | 0         | 1      | 0      | 1      |



All the identified control signals can be generated from **OpCode** bits alone!

# Identifying Main Control Signals



# Implementing Main Control

- Truth Table for Main Control Signals

| Input or output | Signal name | R-format | lw | sw | beq |
|-----------------|-------------|----------|----|----|-----|
| Inputs          | I[6]        | 0        | 0  | 0  | 1   |
|                 | I[5]        | 1        | 0  | 1  | 1   |
|                 | I[4]        | 1        | 0  | 0  | 0   |
|                 | I[3]        | 0        | 0  | 0  | 0   |
|                 | I[2]        | 0        | 0  | 0  | 0   |
|                 | I[1]        | 1        | 1  | 1  | 1   |
|                 | I[0]        | 1        | 1  | 1  | 1   |
| Outputs         | ALUSrc      | 0        | 1  | 1  | 0   |
|                 | MemtoReg    | 0        | 1  | X  | X   |
|                 | RegWrite    | 1        | 1  | 0  | 0   |
|                 | MemRead     | 0        | 1  | 0  | 0   |
|                 | MemWrite    | 0        | 0  | 1  | 0   |
|                 | Branch      | 0        | 0  | 0  | 1   |
|                 | ALUOp1      | 1        | 0  | 0  | 0   |
|                 | ALUOp0      | 0        | 0  | 0  | 1   |



# Single-Cycle Implementation

---

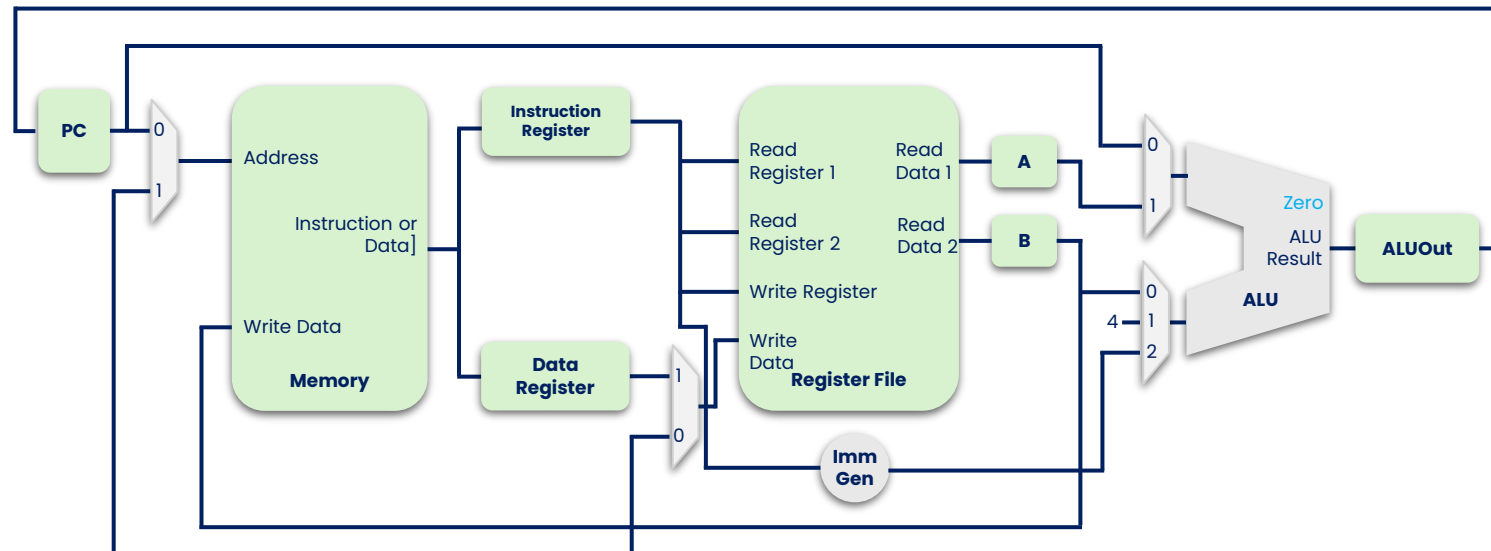
- Drawbacks of single-cycle implementation
  - Very slow clock
    - Must take care of the worst-case!
    - Which instruction takes longest time to execute?
      - Load instruction
        - Uses five functional units in series
        - Instruction Memory → Register File → ALU → Data Memory → Register File
- How can we improve?
  - Make worst-case delay smaller!
  - Cannot improve by improving common case here!
  - Better yet, make it **multicycle** or **pipelined**!

# A Multi-Cycle RISC-V Implementation

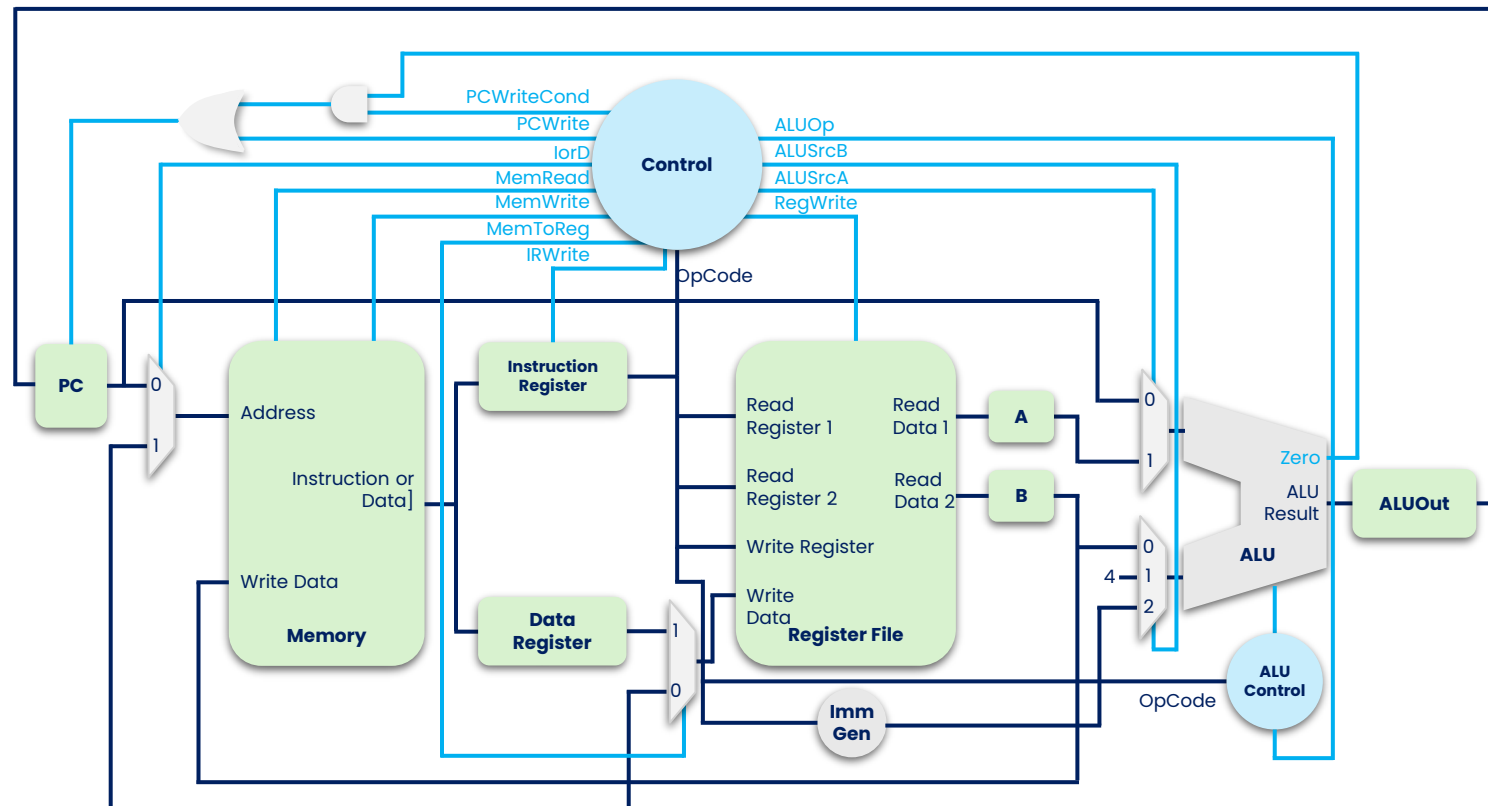


# Multicycle Implementation – Datapath

- Single memory for instructions and data
  - Accessed in different cycles for instruction or data!
- Single ALU for ALU operations, PC+4 and PC+offset!
- Registers to hold result before next cycle of processing



# Multicycle Implementation – Control Signals

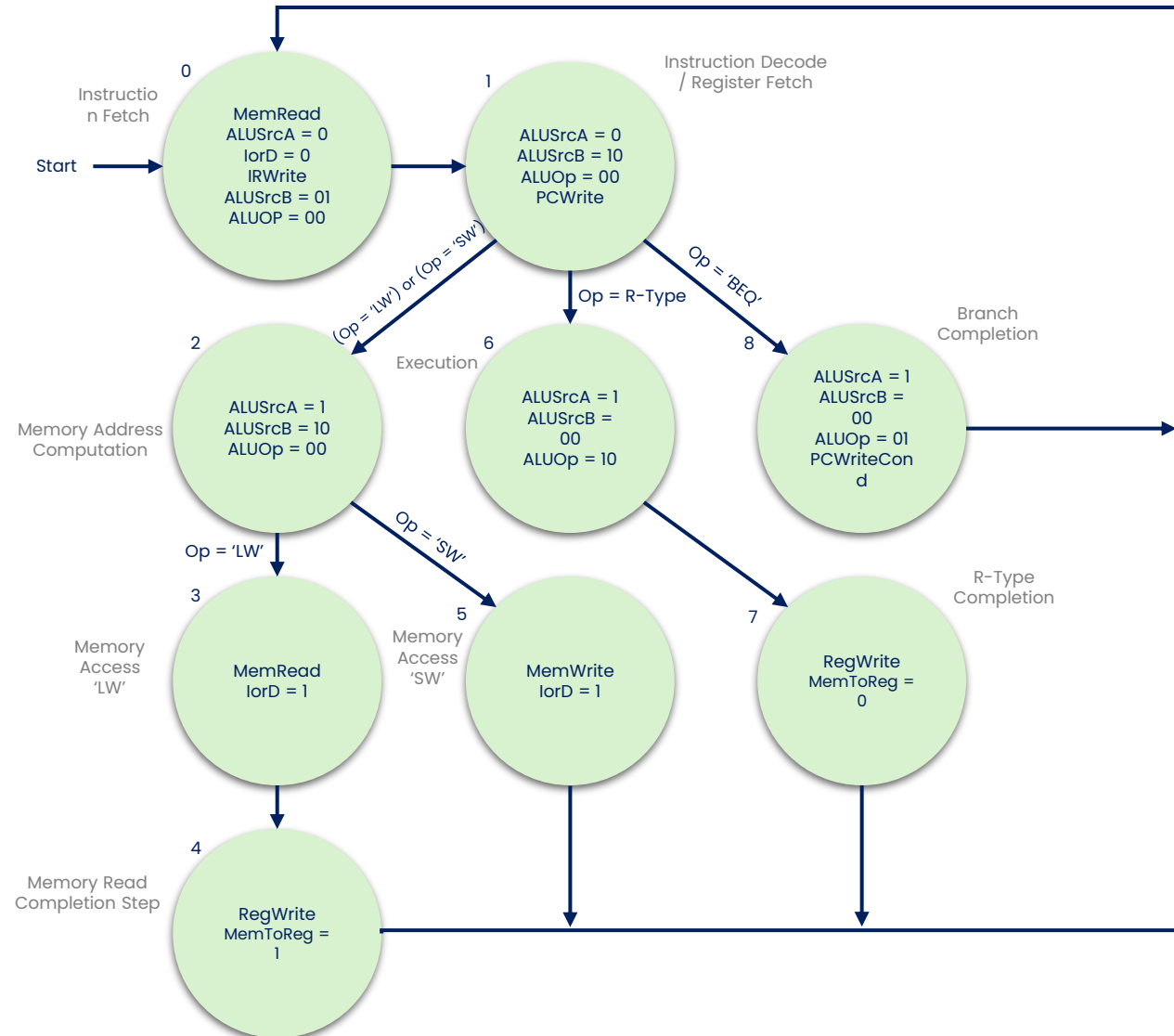


# Multicycle Implementation – Control Logic

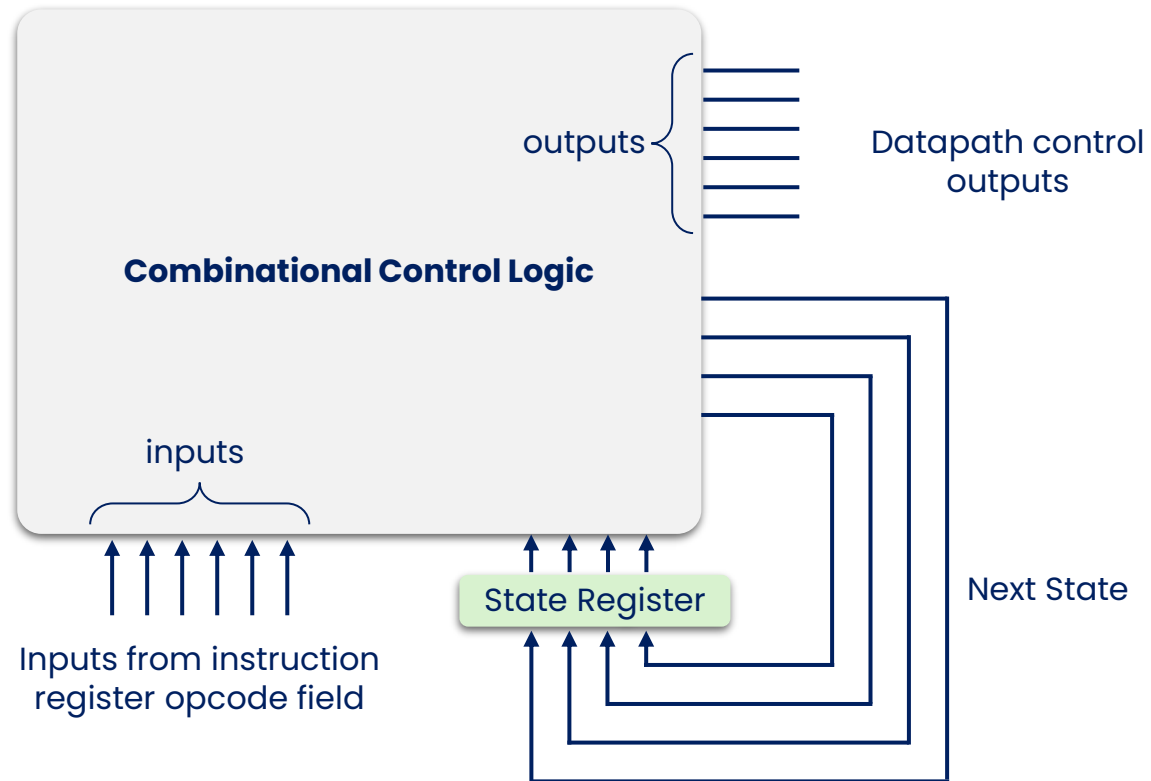
---

- How is the control logic implemented?
  - Combinational circuit just like single-cycle implementation ?
  - No, control signal values are dependent on execution stage!!
    - Need to use a finite state machine (FSM)!

# Multicycle Implementation – FSM for Control Logic



# FSM Implementation



# Benefits of Multicycle Implementation

---

- Clock is faster, cycle time is reduced
  - However, instructions take multiple cycles!
    - But not all instructions have to go through all the stages!
      - This is unlike single-cycle or pipelined implementation where each instruction goes through all the stages!
  - Overall, multicycle is better than single cycle, but not be better than pipelined version which we discuss next!



# Additional Slides

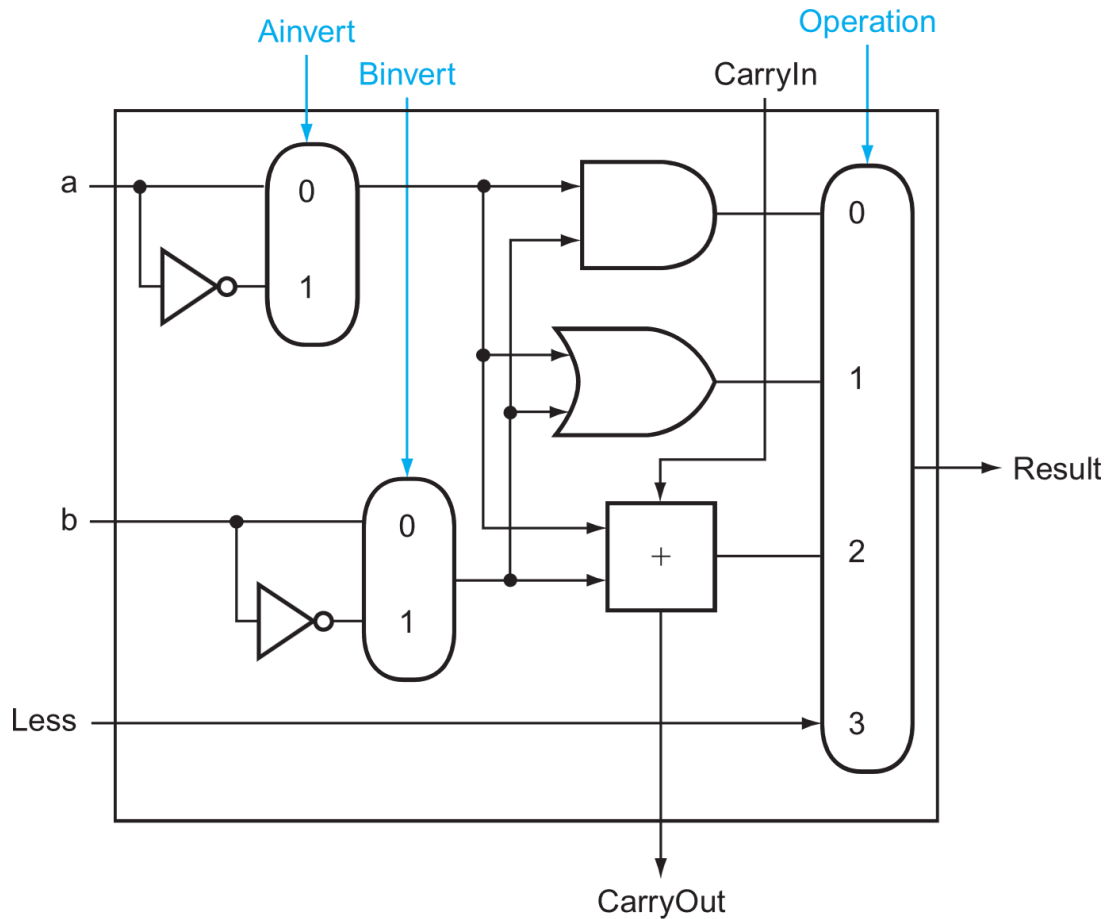


# An implementation of ALU ...



# A Simple 1-bit ALU

- Performs Addition, Subtraction, AND, OR, NOR, NAND ...

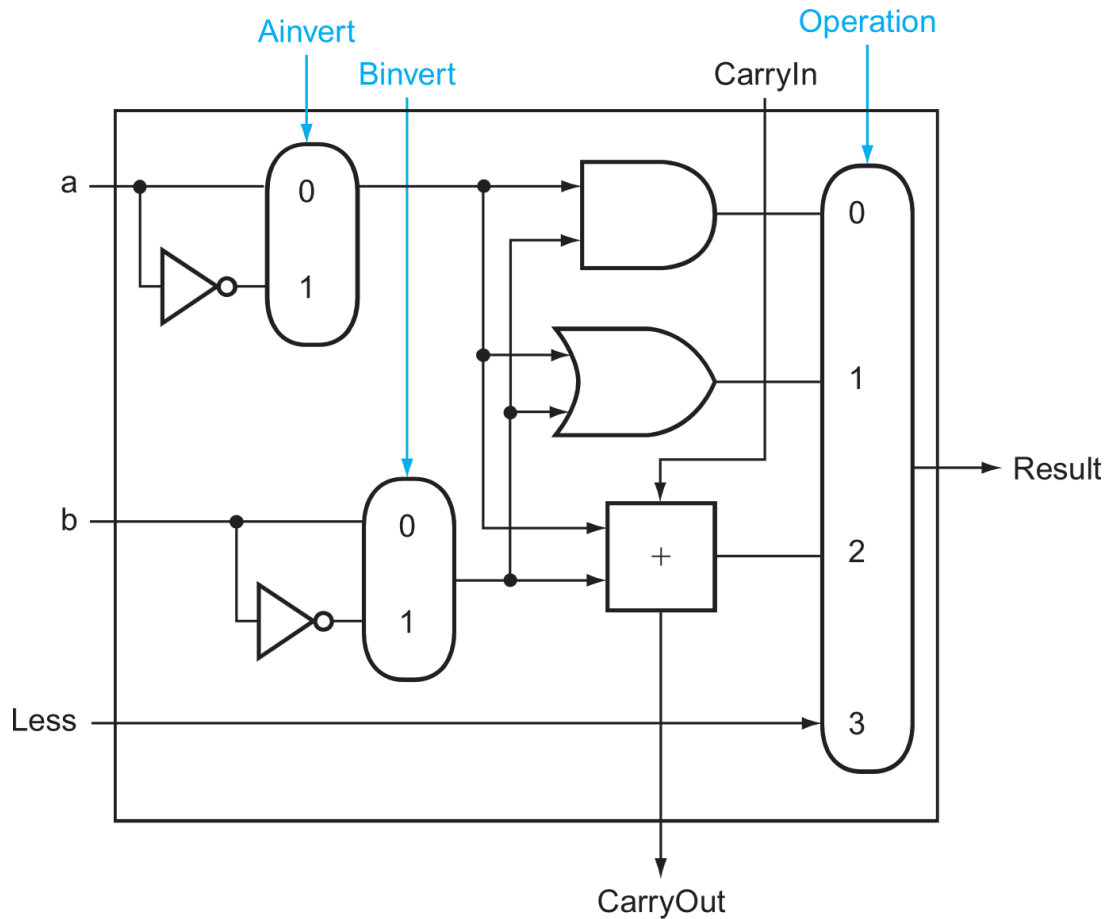


**NOR**  
$$\overline{(a + b)} = \bar{a} \cdot \bar{b}$$

**NAND**  
$$\overline{(a \cdot b)} = \bar{a} \vee \bar{b}$$

# A Simple 1-bit ALU

- Performs Addition, Subtraction, AND, OR, NOR, NAND ...



## Subtraction

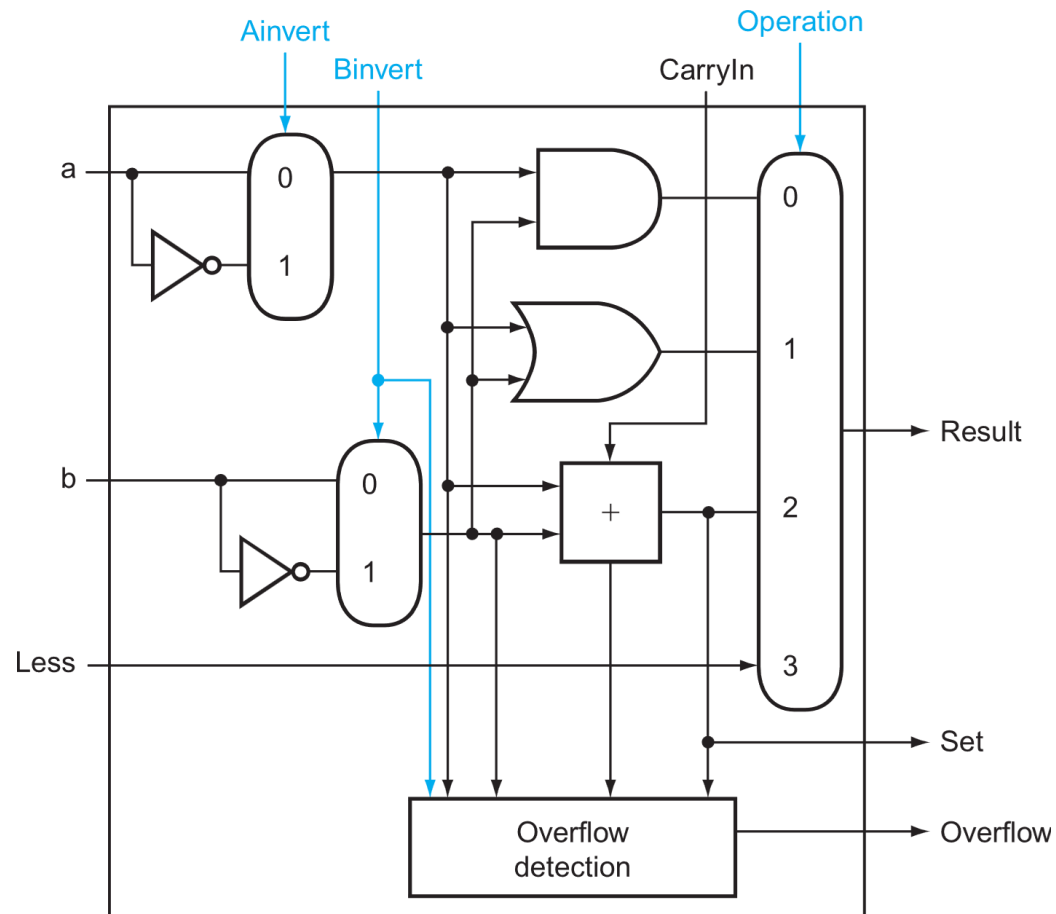
$\text{Binvert} = 1, \text{CarryIn} = 1$

$\text{Result} = a - b$

Binvert and CarryIn can  
be combined as one  
input Bnegate

# Designing **n-bit ALU**

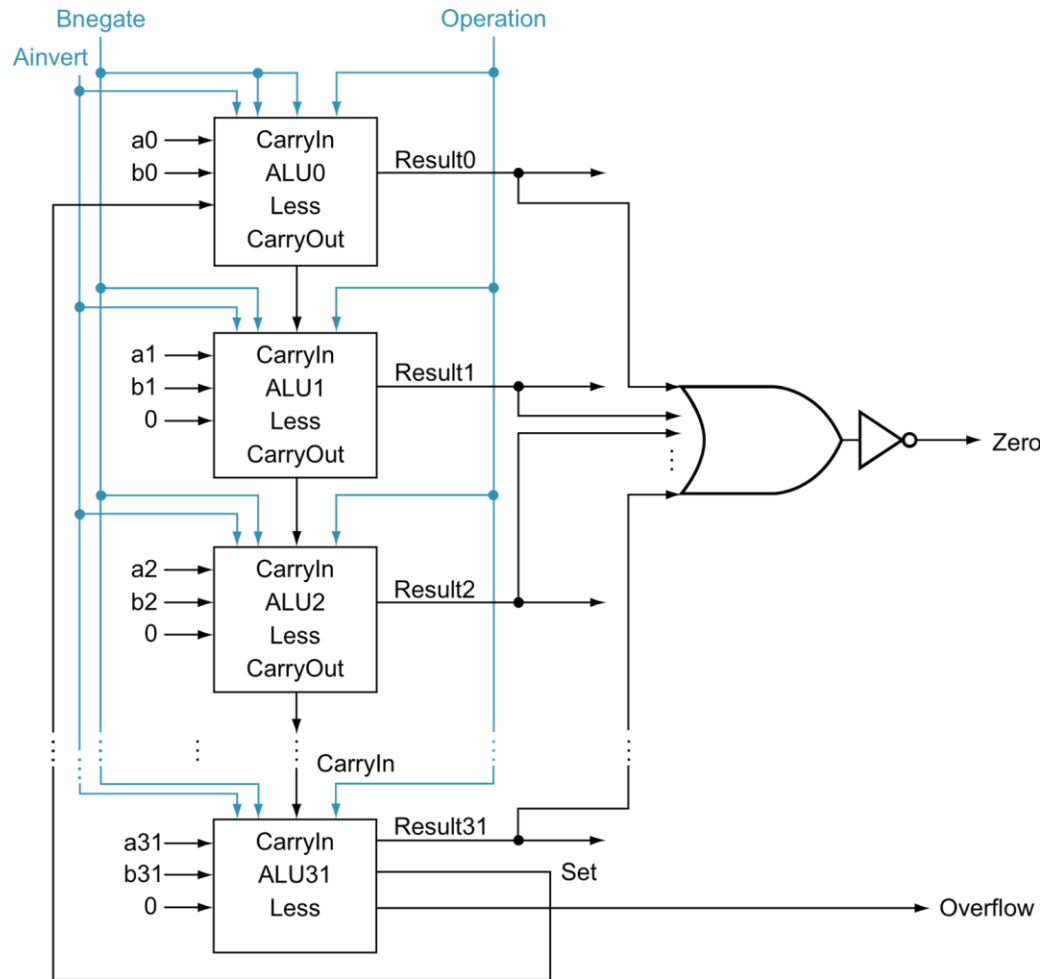
- Can combine  $n$  of 1-bit ALUs to make  $n$ -bit ALU
  - 1-bit ALU for MSB ( $n^{\text{th}}$  bit) is a bit different!



**1-bit ALU for MSB**  
Detects **overflow**  
and **Set** is used as  
**Sign Bit**

# Designing $n$ -bit ALU

- Complete 32-Bit ALU



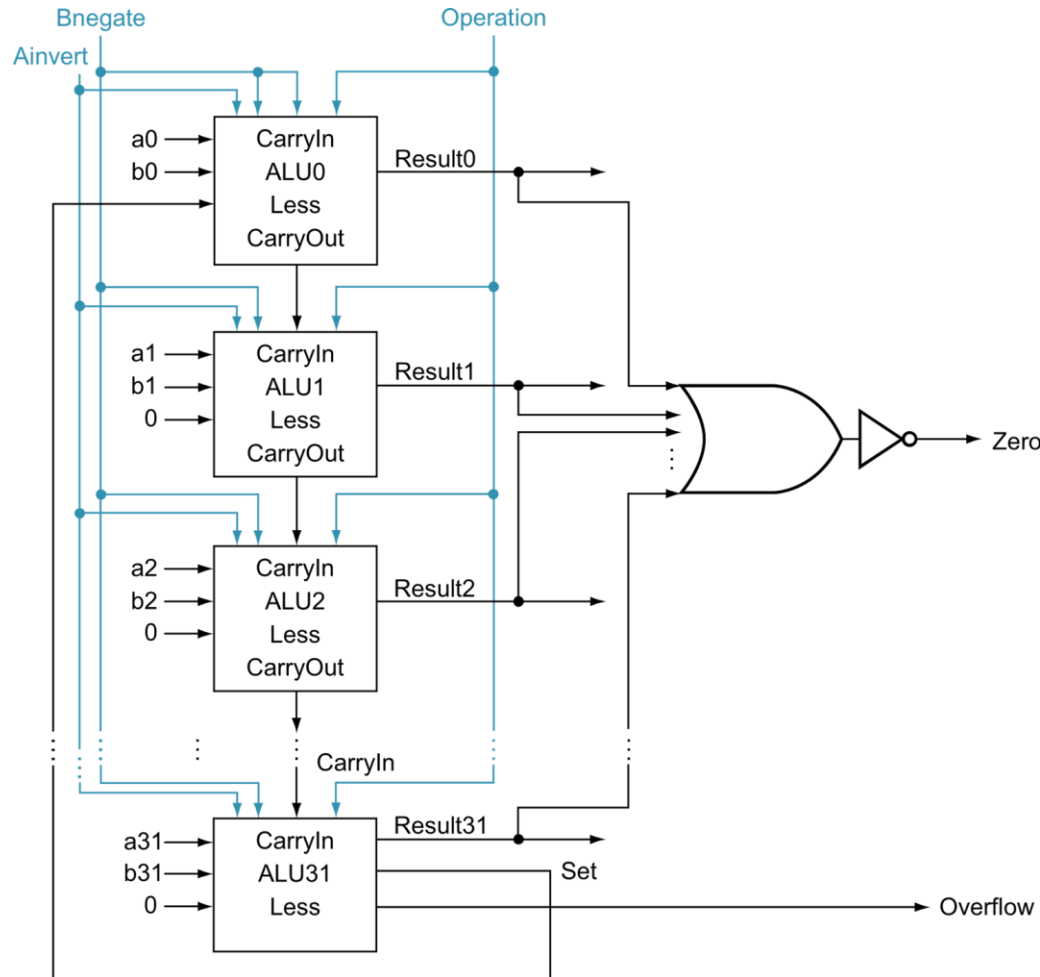
Comparing if  $a < b$   
If  $a < b$ ,  
then  $a - b < 0$

Perform subtraction  
and test Sign Bit (Set)!

If  $a < b$ ,  
 $Result0 = 1$  (= Sign Bit)  
 $Result1, \dots, Result31 = 0$

# Designing $n$ -bit ALU

- Complete 32-Bit ALU

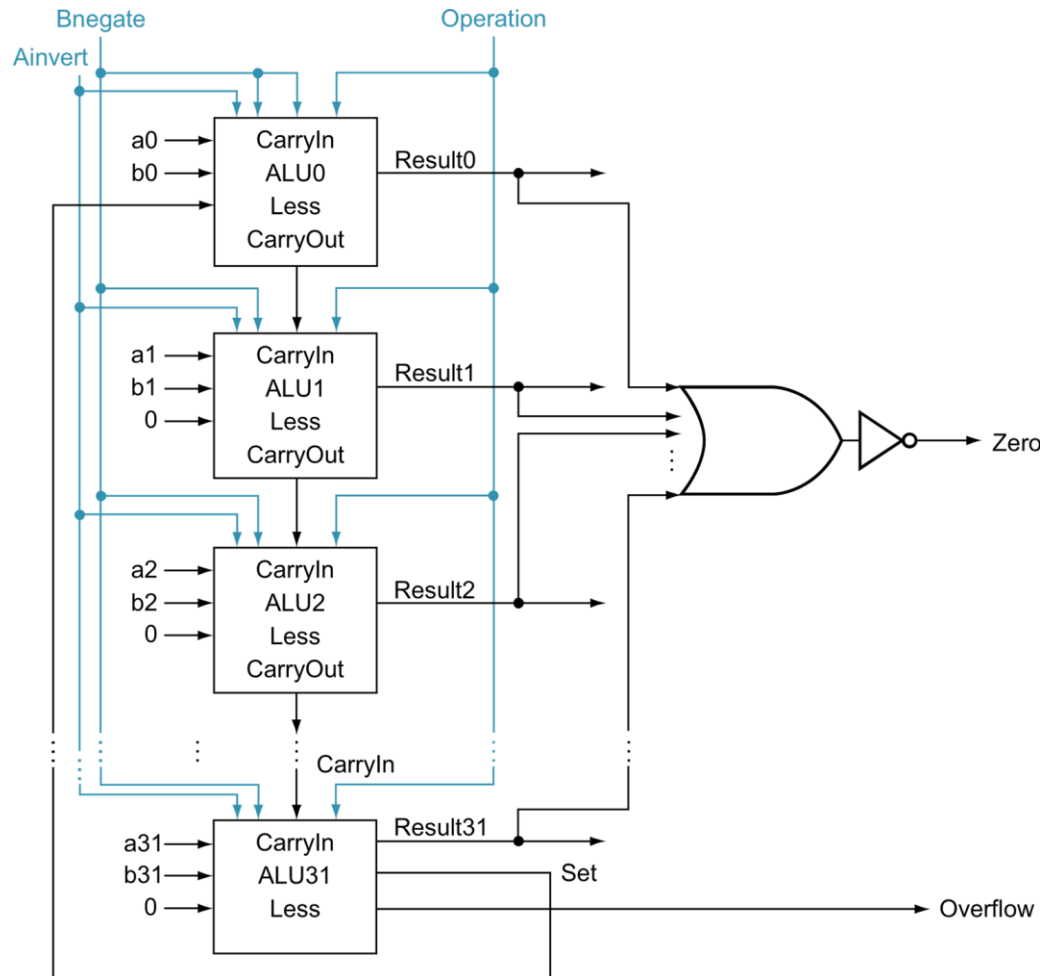


**Comparing if  $a = b$**   
**If  $a = b$ ,**  
**then  $a - b = 0$**

**Perform subtraction**  
**and test if **Result = 0!****

# Designing $n$ -bit ALU

- Controlling Operation of ALU



| ALU control lines | Function      |
|-------------------|---------------|
| 0000              | AND           |
| 0001              | OR            |
| 0010              | add           |
| 0110              | subtract      |
| 0111              | set less than |
| 1100              | NOR           |

**4-bit “ALU Control Lines” is combination of [Ainvert, Bnegate, Operation]**

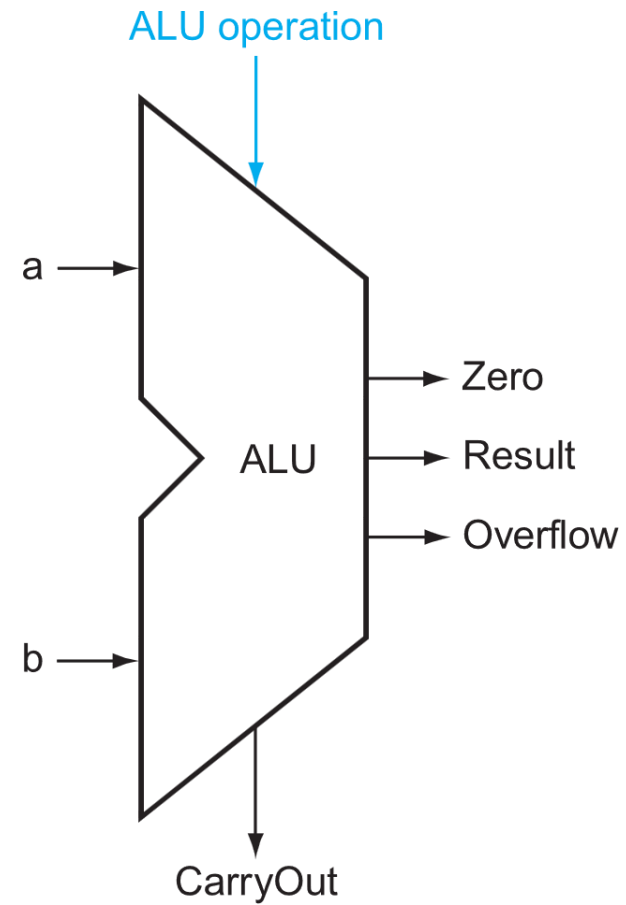
**ALU Control Lines for NAND?**

**1101 i.e.,  $\bar{a} \vee \bar{b}$**



# Designing **n-bit ALU**

- ALU – Top View!



# Overflow in Addition and Subtraction

---

- Overflow occurs when result cannot be correctly represented by number of bits for result
- Unsigned Overflow
  - Can be detected by compiler using branch instructions
    - Addition overflow
      - Sum is less than any of the operands!
    - Subtraction overflow
      - Difference is greater than the operand from which we are subtracting!

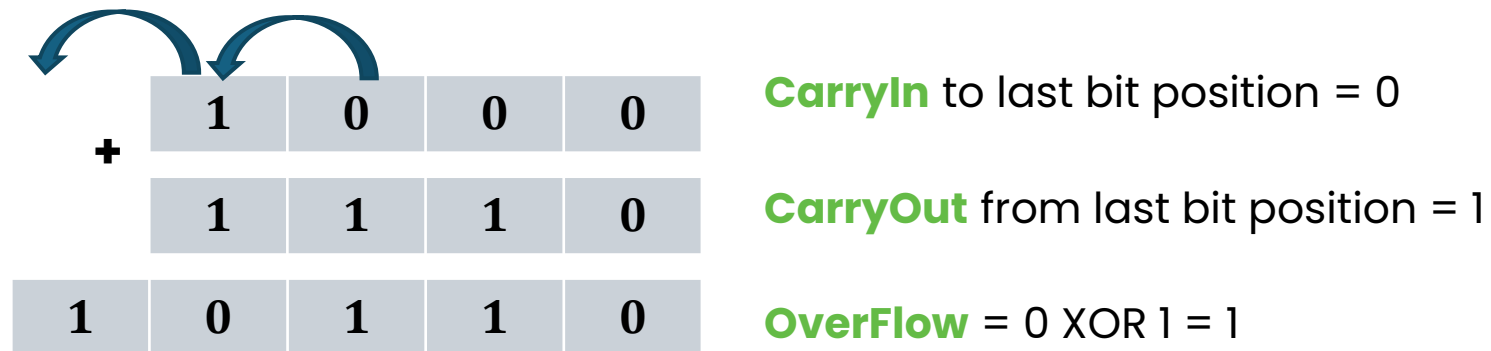
# Overflow in Addition and Subtraction

- Signed Overflow

| Operation | Operand A | Operand B | Result indicating overflow |
|-----------|-----------|-----------|----------------------------|
| $A + B$   | $\geq 0$  | $\geq 0$  | $< 0$                      |
| $A + B$   | $< 0$     | $< 0$     | $\geq 0$                   |
| $A - B$   | $\geq 0$  | $< 0$     | $< 0$                      |
| $A - B$   | $< 0$     | $\geq 0$  | $\geq 0$                   |

# Overflow in Addition and Subtraction

- Hardware to detect signed overflow
  - **Overflow** = XOR of **CarryIn** to last bit position and **CarryOut** from last-bit position
- Example
  - Assume 4-bit operands and 4-bit result!
    - $1000 + 1110 = (-8) + (-2) = -10$



**Result requires 5 bits for correct representation!**

**Thank You**

